



CC1002 - Introducción a la Programación

CLASE 8 : ESTRUCTURAS

Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE

CLASE 08

```
struct Apple {  
  int XX;  
  float radius;  
}
```



Datos Compuestos

Eduardo Godoy (egodoy@dcc.uchile.cl)

Nadia Decar (nadiadecar@ug.uchile.cl)

Si la vida fuera tan simple...

- Int, float, boolean, string...

¡Pero hay más!



Fracciones: Cómo sumar

- ¿Cómo representar una fracción?
- Elemento que contiene dos variables
 - Numerador
 - Denominador

- Representación: $\frac{n}{d}$

- Suma: $\frac{n_1}{d_1} + \frac{n_2}{d_2} = \frac{n_1 * d_2 + n_2 * d_1}{d_1 * d_2}$

Fracciones: Cómo sumar

```
>>> numerador1 = 1
>>> denominador1 = 3
>>> numerador2 = 2
>>> denominador2 = 5
>>> numerador3 = numerador1 * denominador2 + numerador2 * denominador1
>>> denominador3 = denominador1 * denominador2
```

¿Y dónde está la fracción?

```
:::
def sumaFraccion(numerador1 , denominador1 , numerador2, denominador2)
    numerador3 = numerador1 * denominador2 + numerador2 * denominador1
    denominador3 = denominador1 * denominador2
    return ???
```

¿Qué retornar?

¿numerador, denominador... un real?

Estructuras

- Una fracción
 - Es un elemento formado por dos valores
 - Tiene operaciones: suma, multiplicación, ...
- Notar que: `fraccion + fraccion` $\rightarrow$ `fraccion`
- Hay muchos elementos similares:
 - Complejos, polinomios, personas...
- Python permite tener datos compuestos
 - Usaremos el módulo `estructura.py`

Estructuras

- Python permite tener datos compuestos
 - Usaremos el módulo `estructura.py`
- Una estructura permite:
 1. Definir un dato compuesto
 2. Crear un dato compuesto con valores
 3. Conocer los valores de un dato compuesto

Estructura: fraccion

1. Definición

```
>>> import estructura  
>>> estructura.crear("fraccion", "numerador denominador")
```

2. Creación

```
>>> f = fraccion(1, 2)  
⇒ fraccion(numerador=1, denominador=2)
```

3. Valores

```
>>> f.numerador  
⇒ 1  
>>> f.denominador  
⇒ 2
```

Estructura: fraccion

```
import estructura
```

```
estructura.crear("fraccion", "numerador  
denominador")
```

```
def sumaFraccion(f1, f2)
```

```
    numerador = f1.numerador * f2.denominador +  
                f2.numerador * f1.denominador
```

```
    denominador = f1.denominador * f2.denominador
```

```
    f3 = fraccion(numerador, denominador)
```

```
    return f3
```

Estructura: fraccion

```
import estructura  
estructura.crear("fraccion", "numerador  
denominador")  
  
def sumaFraccion(f1, f2)  
    numerador = f1.numerador * f2.denominador +  
                f2.numerador * f1.denominador  
    denominador = f1.denominador * f2.denominador  
    f3 = fraccion(numerador, denominador)  
    return f3
```

Estructura: fraccion

```
import estructura

estructura.crear("fraccion", "numerador
denominador")

def sumaFraccion(f1, f2)
    numerador = f1.numerador * f2.denominador +
                f2.numerador * f1.denominador
    denominador = f1.denominador * f2.denominador
    f3 = fraccion(numerador, denominador)
    return f3
```

Estructura: fraccion

```
import estructura

estructura.crear("fraccion", "numerador
denominador")

def sumaFraccion(f1, f2)
    numerador = f1.numerador * f2.denominador +
                f2.numerador * f1.denominador
    denominador = f1.denominador * f2.denominador
    f3 = fraccion(numerador , denominador)
    return f3
```

Estructura: fraccion

```
import estructura
estructura.crear("fraccion", "numerador denominador")

def sumaFraccion(f1, f2)
    numerador = f1.numerador * f2.denominador +
                f2.numerador * f1.denominador
    denominador = f1.denominador * f2.denominador
    f3 = fraccion(numerador, denominador)
    return f3
```

```
a = fraccion(2, 3)
print(a.numerador)
b = fraccion(5, 4)
```

```
s_ab = sumaFraccion(a, b)
print(s_ab)
```

Estructura: fraccion

```
import estructura
estructura.crear("fraccion", "numerador denominador")

def sumaFraccion(f1, f2)
    numerador = f1.numerador * f2.denominador +
                f2.numerador * f1.denominador
    denominador = f1.denominador * f2.denominador
    f3 = fraccion(numerador, denominador)
    return f3
```

```
a = fraccion(2, 3)
print(a.numerador)
b = fraccion(5, 4)

s_ab = sumaFraccion(a, b)
print (s_ab)
```

Receta de diseño: todo junto

Diseño de la estructura

::

Contrato

::

Propósito

::

Ejemplo:

::

Plantilla

::

Cuerpo de la función

::

Tests

Diseño de la estructura

fraccion: numerador (int) denominador (int)
estructura.crear("fraccion", "numerador denominador")

Contrato

sumaFracciones: fraccion fraccion -> fraccion

Propósito

crear una nueva fraccion que corresponda a la suma
de dos fracciones f1 y f2

Ejemplo:

sumaFracciones(fraccion(1, 2), fraccion(3, 4))
devuelve fraccion(10, 8)

Plantilla

def funcionConFracciones(fraccion1 , fraccion2):
... fraccion1.numerador ... fraccion2.numerador ...
... fraccion1.numerador ... fraccion2. denominador...
... fraccion1. denominador... fraccion2.numerador ...
... fraccion1. denominador... fraccion2. denominador...

Cuerpo de la función

def **sumaFracciones**(f1, f2):
 num = f1.numerador * f2.denominador \
 + f1.denominador * f2.numerador
 den = f1.denominador * f2.denominador
 return fraccion(num, den)

Tests

f12 = fraccion(1, 2)
f34 = fraccion(3, 4)
assert **sumaFracciones**(f12, f34) == **fraccion**(10, 8)

Receta de diseño 1 de 3

Diseño de la estructura

fraccion: numerador (int) denominador (int)

estructura.**crear**("fraccion", "numerador denominador")

Contrato

sumaFracciones: fraccion fraccion -> fraccion

Propósito

crear una nueva fraccion que corresponda a la suma

de dos fracciones f1 y f2

Receta de diseño 2 de 3

Ejemplo:

sumaFracciones(fraccion(1, 2), fraccion(3, 4))

devuelve fraccion(10, 8)

Plantilla

def funcionConFracciones(fraccion1 , fraccion2):

... fraccion1.numerador ... fraccion2.numerador ...

... fraccion1.numerador ... fraccion2. denominador...

... fraccion1. denominador... fraccion2.numerador ...

... fraccion1. denominador... fraccion2. denominador...

Receta de diseño 3 de 3

Cuerpo de la función

def **sumaFracciones**(f1, f2):

 num = f1.numerador * f2.denominador + f1.denominador * f2.numerador

 den = f1.denominador * f2.denominador

return fraccion(num, den)

Tests

f12 = fraccion(1, 2)

f34 = fraccion(3, 4)

assert **sumaFracciones**(f12, f34) == **fraccion**(10, 8)

Receta de diseño: todo junto

Diseño de la estructura

::

Contrato

::

Propósito

::

Ejemplo:

::

Plantilla

::

Cuerpo de la función

::

Tests

Receta de diseño: todo junto

Diseño de la estructura

::

Contrato

::

Propósito

::

Ejemplo:

::

Plantilla

::

Cuerpo de la función

::

Tests

Diseño de la estructura

fraccion: numerador (int) denominador (int)
estructura.crear("fraccion", "numerador denominador")

Contrato

sumaFracciones: fraccion fraccion -> fraccion

Propósito

crear una nueva fraccion que corresponda a la suma
de dos fracciones f1 y f2

Ejemplo:

sumaFracciones(fraccion(1, 2), fraccion(3, 4))
devuelve fraccion(10, 8)

Plantilla

def funcionConFracciones(fraccion1 , fraccion2):
... fraccion1.numerador ... fraccion2.numerador ...
... fraccion1.numerador ... fraccion2.denominador...
... fraccion1. denominador... fraccion2.numerador ...
... fraccion1. denominador... fraccion2. denominador...

Cuerpo de la función

def **sumaFracciones**(f1, f2):
 num = f1.numerador * f2.denominador \
 + f1.denominador * f2.numerador
 den = f1.denominador * f2.denominador
 return fraccion(num, den)

Tests

f12 = fraccion(1, 2)
f34 = fraccion(3, 4)
assert **sumaFracciones**(f12, f34) == **fraccion**(10, 8)

Una estructura no es mutable

```
f1 = fraccion(4, 3)
```

```
f1.numerador
```

→ 4

```
f1.numerador = 5   # Error!
```

```
f1 = fraccion(5, 3)   # OK!
```